

¿Cuánto vale un diamante?

Mejora iterativa de un modelo de valoración y dashboard
interactivo en Dash

Actividad 6 — Proyecto final, etapa de transferencia

Programación para Ciencia de Datos II

Presentado por

Brandow Brusly León Rodríguez

Fundación Universitaria Compensar

Facultad de Ingeniería

Ingeniería en Ciencia de Datos

Bogotá D.C. — 24 de septiembre de 2026

1. El problema y de dónde parto

Desde la práctica 2 trabajo el mismo problema: **valoración automática de diamantes a partir de los atributos del catálogo** (peso, corte, color, pureza y medidas), sobre el conjunto `diamonds` de 53.940 referencias. La pregunta de negocio es la de una joyería que compra y vende piedras: *¿se puede fijar un precio justo sin depender del criterio de un tasador, con un error que un comprador aceptaría?*

El recorrido anterior dejó tres cosas establecidas:

- **Práctica 2.** Una recta con el peso explica cerca del 85 % de la varianza del precio, pero tiene intercepto negativo: predice precios imposibles para las piedras pequeñas.
- **Actividad 4.** La regresión múltiple con las continuas (peso, profundidad, tabla) apenas mejoró la recta. El contraste de hipótesis destapó una **paradoja de Simpson**: globalmente el color J (el peor) cuesta más que el D (el mejor), porque los J son más pesados; en una banda de peso comparable la relación se invierte.
- La conclusión fue que las continuas estaban agotadas y que el siguiente paso era trabajar en escala logarítmica con las variables categóricas.

Este informe documenta esa siguiente etapa: la evaluación del modelo de partida, seis iteraciones del modelo con experimentación controlada de hiperparámetros, la regularización, el efecto de la calidad y cantidad de los datos, y el dashboard que presenta los hallazgos. Cada decisión va acompañada de la evidencia que la justificó.

2. Evaluación del modelo de partida

2.1. Método de evaluación, fijo para todas las iteraciones

Para que las iteraciones fueran comparables, fijé el protocolo antes de tocar el modelo: partición 70–30 con semilla 42 (37.744 filas de entrenamiento y 16.176 de prueba), selección de hiperparámetros **solo** con validación cruzada de 5 pliegues sobre el entrenamiento, y el conjunto de prueba reservado para medir cada modelo una única vez. Todas las métricas se reportan en dólares, aunque el modelo se entrene en escala logarítmica.

Usé tres métricas. R^2 y RMSE permiten comparar con las actividades anteriores. La métrica de negocio es el **MAPE** (error porcentual absoluto medio): para una joyería, equivocarse en 500 USD es grave en una piedra de 600 y despreciable en una de 15.000. El MAPE captura esa asimetría y el R^2 no.

2.2. Diagnóstico

El modelo de la actividad 4 (M1: $\text{precio} \sim \text{peso} + \text{profundidad} + \text{tabla}$), medido con este protocolo, da $R^2 = 0,8587$, $\text{RMSE} = 1,484$ USD y un **MAPE de 38,0 %**. Además produce **894 precios negativos** en el conjunto de prueba. Las cifras difieren levemente de las de la actividad 4 ($R^2 = 0,8532$) porque aquí se eliminaron antes de la partición los 20 registros con dimensiones en cero, y eso cambia qué filas caen en cada subconjunto.

Un error típico del 38 % no sirve para fijar precios. Me pregunté si había que replantear la pregunta o mejorar el modelo. La respuesta fue **mantener la pregunta y replantear la forma del modelo**. El diagnóstico de residuos de la actividad 4 mostraba un embudo, es decir, un error que crece con el precio, y una curva sistemática. Esos son síntomas de una relación multiplicativa modelada como aditiva, no de que la pregunta esté mal hecha. Lo único que ajusté de la pregunta fue el criterio de éxito: pasó a ser el error porcentual, porque así se mide una tasación en la práctica.

3. Iteraciones del modelo

La Tabla 1 resume las seis iteraciones. Todas se ejecutan con el script `modelo.py`, que es determinista: con los mismos datos produce exactamente las mismas cifras.

Tabla 1. Iteraciones del modelo, medidas en el conjunto de prueba (16.176 diamantes).

	Modelo	R^2	RMSE (USD)	MAPE	Negativos
M0	Recta $\text{precio} \sim \text{peso}$ (práctica 2)	0,8541	1.509	37,7 %	462
M1	Múltiple con continuas (actividad 4)	0,8587	1.484	38,0 %	894
M2	Peso + corte + color + pureza (one-hot), en USD	0,9194	1.121	44,0 %	1.452
M3	$\log(\text{precio}) \sim \log(\text{peso}) + \text{categóricas}$	0,9612	778	10,4 %	0
M4	M3 + interacciones de grado 2 + Ridge ($\alpha = 3,16$)	0,9655	733	9,5 %	0
M5	Gradient boosting sobre $\log(\text{precio})$, <code>max_depth</code> = 6	0,9816	535	6,5 %	0

M2: las categóricas solas no bastan. Agregar corte, color y pureza con codificación one-hot sube el R^2 a 0,9194. Sin embargo, **el MAPE empeora** (de 38,0 a 44,0 %) y los precios negativos suben a 1.452. En escala aditiva, las primas de calidad se suman en dólares a todas las piedras por igual. Un diamante de 0,3 quilates con pureza IF recibe la misma prima absoluta que uno de 2 quilates, así que las piedras baratas quedan muy mal valoradas. Este es el caso que justifica usar el MAPE: el R^2 premiaba un modelo que para el negocio era peor.

M3: ingeniería de características log-log. La corrección fue transformar precio y peso a logaritmo. Así, cada atributo de calidad actúa como un *porcentaje* sobre el precio y no como una

suma fija, que es como se comporta el mercado. El error típico cae de 38,0 a **10,4 %** y los precios negativos desaparecen por construcción. Es el salto más grande de todo el proyecto, y no vino de un algoritmo más sofisticado sino de entender la forma del problema. M3 es además interpretable: la elasticidad del precio respecto al peso es **1,88**, es decir, 1 % más de peso sube el precio un 1,88 %.

M4 y M5. Se tratan en la sección 4: son la experimentación con hiperparámetros y la regularización.

4. Experimentación con hiperparámetros y regularización

4.1. Criterio de selección: regla de un error estándar

En los dos barridos apliqué la misma regla: calcular el R^2 de validación cruzada para cada valor del hiperparámetro y quedarme con el **modelo más simple cuyo R^2 esté a menos de un error estándar del mejor**. Escoger el máximo absoluto premia diferencias que están dentro del ruido de la validación y empuja hacia modelos más complejos sin evidencia. Cada barrido varía una sola cosa y deja el resto fijo, como en un experimento controlado.

4.2. Regularización: Ridge y Lasso sobre M3 con interacciones

M4 amplía M3 con todas las interacciones de grado 2 (189 términos): peso×color, color×pureza, etc. Con tantos términos existe riesgo de sobreajuste, así que la regresión se penaliza con Ridge (L2) y Lasso (L1), barriendo α en escala logarítmica.

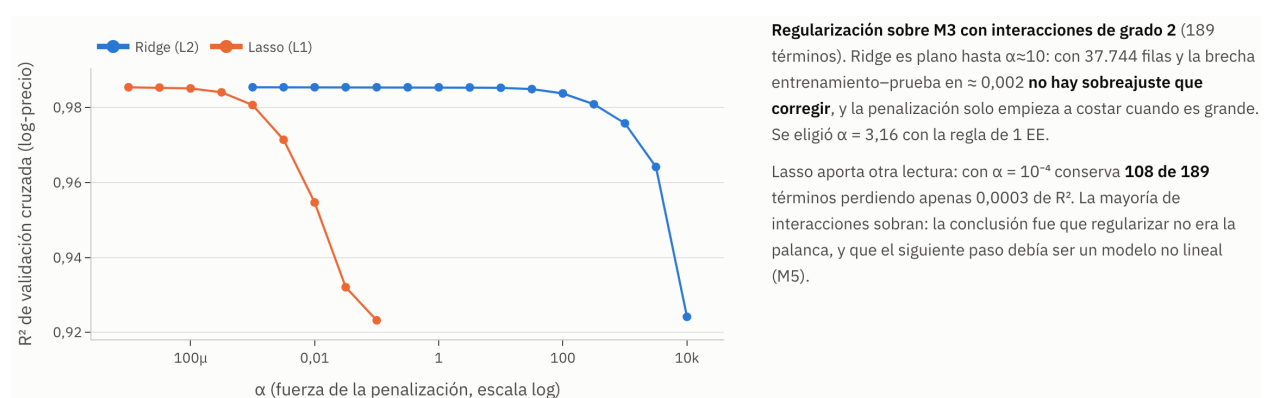


Figura 1. Barrido de α para Ridge y Lasso (captura del dashboard, pestaña 4).

Resultado: no había sobreajuste que corregir. La curva de Ridge es plana hasta $\alpha \approx 10$ y solo empieza a caer con penalizaciones grandes. Con 37.744 filas para 189 términos, la brecha entre entrenamiento y validación es de dos milésimas. La regla de un error estándar eligió $\alpha = 3,16$

y M4 mejora a M3 en medio punto de R^2 . Lasso aportó otra lectura: con $\alpha = 10^{-4}$ conserva solo **108 de los 189 términos** con una pérdida despreciable de R^2 . La mayoría de interacciones sobran. Conclusión: *la regularización no era la palanca*. El modelo lineal había llegado a su techo, y para bajar el error hacía falta un modelo que aprendiera las interacciones por sí mismo.

4.3. Profundidad del gradient boosting

M5 es un `HistGradientBoostingRegressor` sobre $\log(\text{precio})$ con todas las variables, incluidas las medidas x, y, z . El hiperparámetro que controla el sobreajuste es la profundidad de cada árbol. La barrí de 1 a 12 con el resto fijo: 300 árboles y tasa de aprendizaje 0,1.

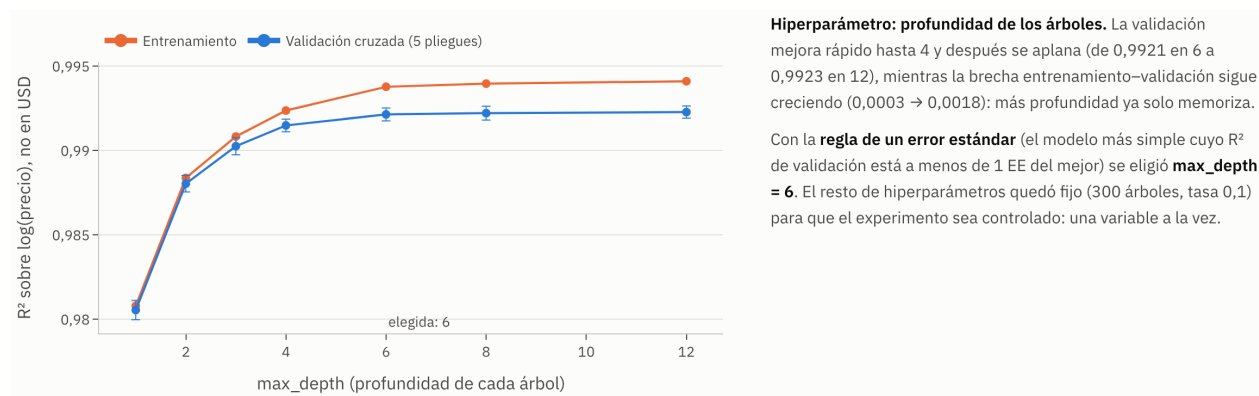


Figura 2. R^2 de entrenamiento y de validación cruzada según la profundidad (pestaña 4 del dashboard).

La validación sube rápido hasta profundidad 4 y después se aplana: de 0,9921 en 6 a 0,9923 en 12. La brecha entre entrenamiento y validación, en cambio, sigue creciendo, de 0,0003 a 0,0018. Pasada esa zona, más profundidad solo memoriza. La regla de un error estándar eligió $\text{max_depth} = 6$. En prueba, M5 alcanza $R^2 = 0,9816$, RMSE de 535 USD y **MAPE de 6,5 %**, con una brecha entrenamiento-prueba de 0,0047: el modelo generaliza.

5. Calidad y cantidad de los datos

5.1. Una característica nueva y 20 registros defectuosos

El catálogo trae 20 piedras con alguna dimensión en cero, algo físicamente imposible. Probé una característica nueva, $\log(\text{volumen}) = \log(x \cdot y \cdot z)$, de dos formas. La primera fue imputar los ceros con la mediana, que es el atajo habitual: el R^2 **bajó** de 0,9612 a 0,9592. La segunda fue eliminar esos 20 registros: el R^2 subió apenas a 0,9613. Veinte filas defectuosas entre 53.940 (0,04 %) bastaron para anular la ganancia de una característica. Por eso la limpieza se

aplica antes de la partición en todos los modelos. El volumen resultó casi redundante con el peso, porque la densidad del diamante es constante, así que M3 no lo incluye.

5.2. ¿Conseguir más datos ayudaría?

Reentrené M3 y M5 con fracciones crecientes del entrenamiento (del 2 % al 100 %) y los medí siempre contra el mismo conjunto de prueba.

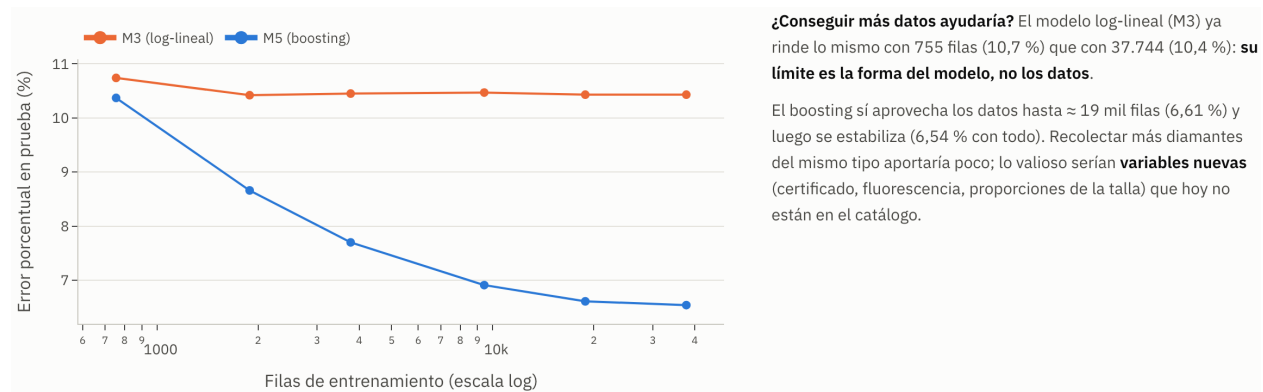


Figura 3. Curva de aprendizaje: error porcentual en prueba según el tamaño del entrenamiento.

M3 rinde lo mismo con 755 filas (10,7 %) que con 37.744 (10,4 %): **su límite es la forma del modelo, no la cantidad de datos.** M5 aprovecha los datos hasta unas 19 mil filas (6,61 %) y luego se estabiliza. Recolectar más diamantes del mismo tipo aportaría poco. Lo valioso sería recolectar **variables nuevas**: laboratorio certificador, fluorescencia, proporciones detalladas de la talla. Esa es la recomendación de recolección para la siguiente versión.

6. Fiabilidad del modelo final

Para decidir si el modelo es fiable no basta un R^2 alto. Revisé tres cosas:

1. **Generalización.** La brecha entre entrenamiento y prueba es de 0,0047 en R^2 , y el gráfico de predicho frente a real (pestaña 4) sigue la diagonal en todo el rango de precios, sin la curva ni el embudo de M1.
2. **Incertidumbre honesta.** El tasador da un intervalo del 80 % en lugar de un número suelto. Calibré el intervalo con los residuos *fuera de pliegue* del entrenamiento (`cross_val_predict`) y medí su cobertura en prueba: **80,1 %**, que coincide con el valor nominal. El intervalo va de $-9,9 \%$ a $+10,6 \%$ alrededor de la estimación. En una primera versión calibré el intervalo con los residuos de prueba, pero así la cobertura da 80 % por construcción y no demuestra nada. Lo corregí antes de publicar.

3. **Coherencia con el dominio.** Los efectos que estima M3 son monótonos: cada escalón de color y de pureza suma prima. Esto resuelve la paradoja de Simpson: a igual peso, corte y pureza, **D vale un 67 % más que J**, aunque en bruto D promedie 3.168 USD y J 5.324 USD (los J pesan en promedio 1,16 quilates y los D 0,66).

Con esta evidencia doy el modelo por fiable para su propósito: una primera tasación automática con ± 10 % de incertidumbre, que un tasador humano revisa solo cuando el precio de lista se sale del intervalo.

7. El dashboard

7.1. Decisiones de diseño

- **Tecnología.** Dash con Plotly y Pandas, según la guía de la actividad. El dashboard (`app.py`) **no entrena nada al arrancar**: lee los resultados que deja `modelo.py` en `resultados/`. Separar el cálculo de la presentación hace que la carga en Binder tarde segundos y no los tres minutos del entrenamiento.
- **Estructura narrativa.** Cinco pestañas que siguen el hilo del informe: la historia, explorar el catálogo, la paradoja del color, los experimentos del modelo, y el tasador con el detector de oportunidades. Cada gráfico va acompañado de un texto que dice qué mirar y qué concluir.
- **Color con función.** Corte, color y pureza son variables ordinales, así que se codifican con una rampa de un solo tono (azul claro = peor calidad, azul oscuro = mejor) y no con colores arbitrarios. Las comparaciones entre dos series (entrenamiento frente a validación, Ridge frente a Lasso) usan azul y naranja, un par que se distingue también con daltonismo. En los gráficos de barras, el gris es contexto y el azul marca lo que se debe mirar.
- **Tipografía.** Spectral, una serifa de libro, para títulos y cifras, que evoca un certificado de tasación; IBM Plex Sans para el texto y los controles, con números tabulares en indicadores y tablas.
- **Formato local.** Coma decimal y punto de miles en ejes, tablas y textos.
- **Rendimiento.** El gráfico de dispersión usa WebGL (`Scattergl`) y, si el filtro supera 12.000 puntos, dibuja una muestra aleatoria. El panel lo avisa en su resumen.



Figura 4. Pestaña 1: indicadores y evolución del error en cada iteración.

7.2. Interactividad

- **Explorar el catálogo.** Filtro por corte (selección múltiple), rango de peso, variable de color (corte, color o pureza) y escala lineal o logarítmica. Al cambiar la escala se ve por qué se eligió la transformación log-log: la nube se convierte en una banda recta.
- **La paradoja del color.** El usuario elige dos colores y una banda de peso, y el panel recalcula en vivo la diferencia de precios. Si la banda es estrecha, el confusor desaparece y aparece la prima real del color.
- **Experimentos.** Un selector recorre los cuatro experimentos: profundidad, regularización, cantidad de datos y calidad de datos. Cada uno trae su gráfico y su interpretación. Otro selector compara el predicho frente a real de M3 y de M5.
- **Tasador.** Peso, corte, color y pureza producen el precio estimado, su intervalo del 80 % y un análisis de sensibilidad: cuánto cambia el precio al mover un solo atributo.

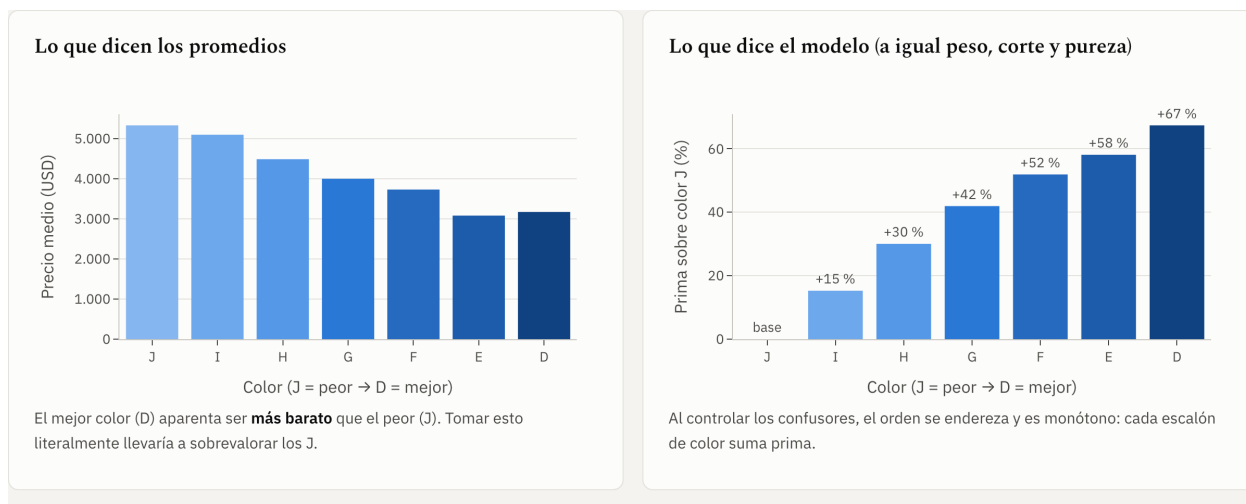


Figura 5. Pestaña 3: lo que dicen los promedios frente a lo que dice el modelo.

7.3. Aporte original: el detector de oportunidades

El modelo no solo tasa: también encuentra **piedras malpreciadas**. El detector recorre los diamantes del conjunto de prueba, que el modelo nunca vio, y lista los que tienen un precio de lista muy por debajo de lo que vale su combinación de atributos, filtrando por presupuesto y descuento mínimo. En todo el conjunto de prueba hay 255 diamantes a más de 20 % por debajo de su valor estimado. Para una joyería compradora es una lista de revisión; para una vendedora, una alerta de precios mal fijados. Los desvíos extremos (más del 50 %) se señalan para revisión manual, porque pueden ser errores de captura en el catálogo.

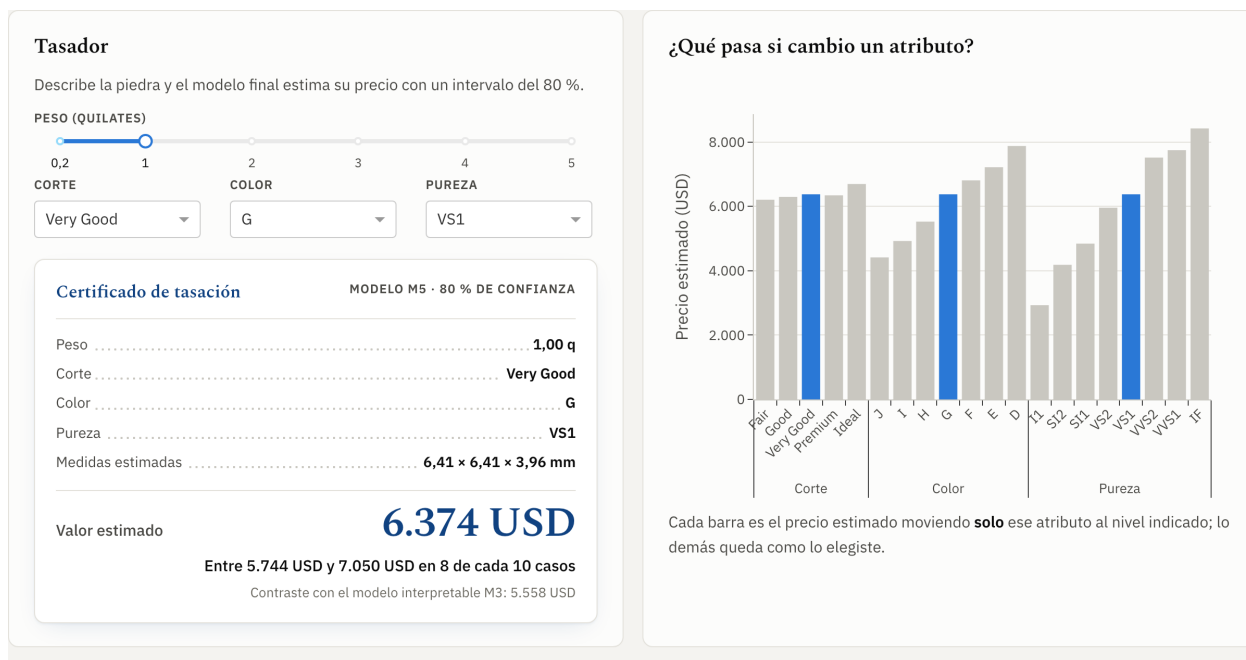


Figura 6. Pestaña 5: tasador con intervalo y análisis de sensibilidad.

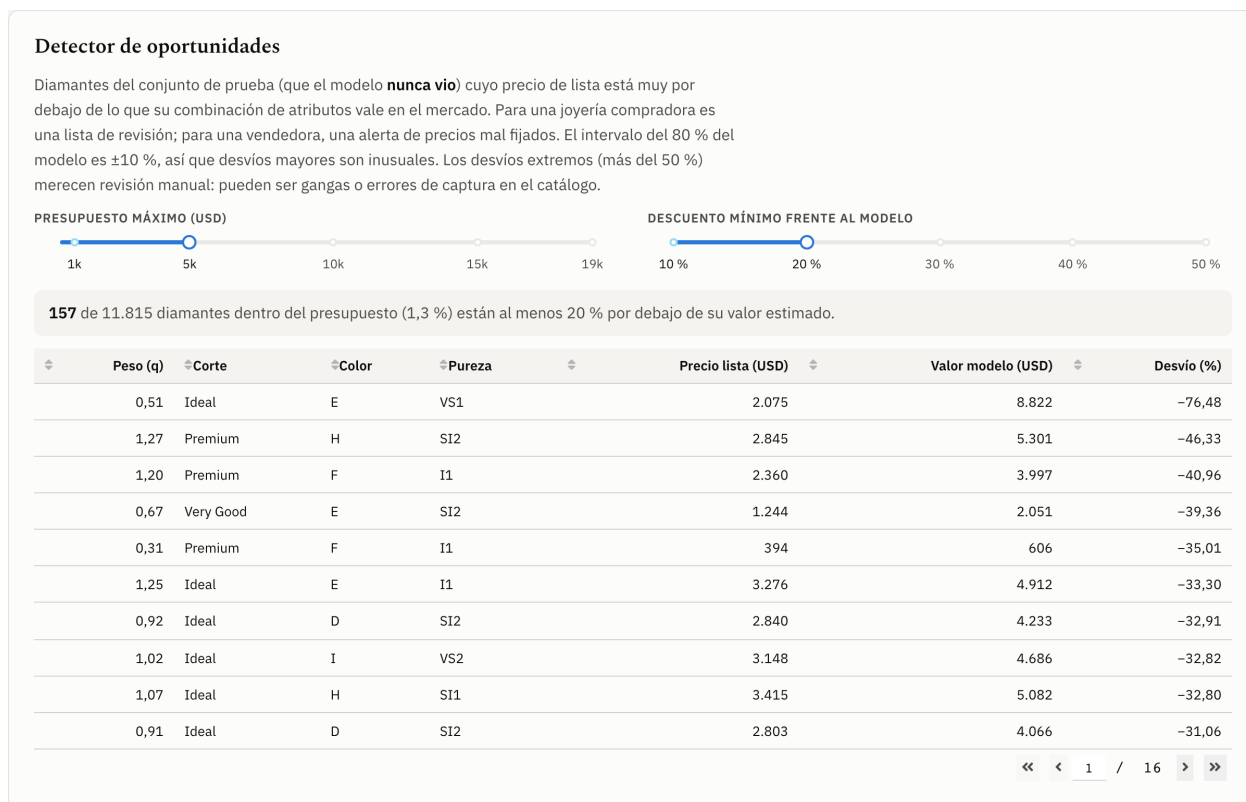


Figura 7. Pestaña 5: detector de oportunidades.

7.4. Pruebas

- **Prueba en navegador real.** Un script de Playwright abre el dashboard, recorre las cinco pestañas, acciona los selectores (color, escala, experimentos, modelo) y comprueba que no haya errores de consola ni *callbacks* fallidos. Resultado: 0 errores.
- **Prueba en otro entorno.** Antes de publicar simulé Binder en local (Jupyter Server con `jupyter-server-proxy` y el mismo script `binder/start`) y repetí la prueba completa a través del proxy. Después verifiqué el enlace real en `mybinder.org`, que construye la imagen desde cero en otra máquina con las versiones fijadas en `requirements.txt`.
- **Verificación de cifras.** El script `verificar.py` recalcula desde el CSV crudo y los modelos guardados cada cifra citada en este informe y en el dashboard; M0 y M1 los recalcula con NumPy puro, sin scikit-learn. Resultado: **40 comprobaciones, 0 incorrectas**. En su primera ejecución detectó una cifra mal transcrita (el MAPE de M2), que se corrigió.
- **Revisión visual.** La revisión de las capturas encontró problemas que se corrigieron: un eje truncado que exageraba diferencias mínimas (se cambió por un gráfico de puntos), etiquetas recortadas y separadores decimales en formato inglés.
- **Crítica de diseño y accesibilidad.** Una revisión heurística (heurísticas de Nielsen: 27/40 en la primera versión), un detector automático de antipatronos visuales y una auditoría con axe-core encontraron cinco problemas prioritarios, que se corrigieron:
 - las pestañas no se podían usar con teclado; ahora se recorren con Tab y las flechas;
 - la página no declaraba el idioma;
 - en el celular la tabla desbordaba la pantalla;
 - los filtros vacíos dejaban gráficos rotos; ahora muestran un mensaje que dice qué hacer;
 - algunos textos no alcanzaban el contraste AA.

Además, el resultado del tasador se rediseñó como un *certificado de tasación* y se agregó una guía de las escalas gemológicas (FairIdeal, JD, I1IF) y de las métricas. Tras los cambios, axe no reporta fallos de contraste, y los avisos que quedan pertenecen a componentes internos de Dash (desplegables, deslizadores y paginación de la tabla).

7.5. Manual de uso

1. **Sin instalar nada:** abrir el enlace de Binder del archivo `enlaces.txt`. La primera carga puede tardar unos minutos mientras Binder prepara el entorno; el dashboard aparece directamente.

2. **En un equipo local:** con Python 3.12, `pip install -r requirements.txt` y luego `python app.py`. El dashboard queda en `http://127.0.0.1:8050`.
3. **Para regenerar los resultados:** `python modelo.py` (unos 3 minutos) y luego `python verificar.py`.

8. Conclusiones

Sobre el modelo. En seis iteraciones el error típico de tasación pasó de 38,0 % a **6,5 %** (R^2 de 0,8587 a 0,9816), sin predicciones negativas y con un intervalo del 80 % cuya cobertura se verificó en datos no vistos. El salto decisivo fue entender que el precio es multiplicativo (transformación log-log), no el algoritmo. La regularización se probó con rigor y resultó innecesaria, porque el modelo lineal no sobreajustaba. El boosting ganó porque aprende interacciones que el modelo lineal solo captura a medias.

Sobre el problema. El mercado paga el peso de forma más que proporcional (elasticidad 1,88). La pureza pesa más que el color, y el color más que el corte: pasar de I1 a IF triplica el precio (+208 %), mientras que el mejor corte frente al peor solo suma un 18 %. El color sí se paga (D vale un 67 % más que J a igualdad del resto), aunque los promedios brutos digan lo contrario. Para un comprador, el corte es donde más se ahorra; para un vendedor, el detector de oportunidades señala precios mal fijados.

Planteamientos para seguir. (1) Recolectar variables nuevas (certificado, fluorescencia, proporciones), porque la curva de aprendizaje muestra que más filas del mismo tipo ya no mejoran el modelo. (2) Validar con precios actuales, porque el conjunto de datos es histórico. (3) Pedir las medidas reales en el tasador en lugar de estimarlas a partir del peso, como hace hoy la interfaz.

Referencias

- Oehlert, G. W. (2010). *A First Course in Design and Analysis of Experiments*. University of Minnesota.
- Plotly. (2023, 30 de agosto). *Dash Documentation*. <https://dash.plotly.com/>
- The Turing Way Community. (2021). *Zero to Binder*. <https://the-turing-way.netlify.app/communication/binder/zero-to-binder.html>
- Kohavi, R., Longbotham, R., Sommerfield, D., & Henne, R. M. (2008). Controlled experiments on the web: survey and practical guide. *Data Mining and Knowledge Discovery*, 18(1), 140–181.

- Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning* (2.ª ed.). Springer. (Regla de un error estándar, §7.10.)
- Pedregosa, F. et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Wickham, H. (2016). *ggplot2: Elegant Graphics for Data Analysis*. Springer. (Conjunto de datos diamonds.)